

ТОО "Колледж Хекслет"

Лабораторная работа №5

Работа с файловой системой — асинхронная.

Выполнено студентом:
Дубинин Кирилл

Общие требования:

- Использовать модуль `fs.promises` (или `require('fs/promises')`).
- Весь код должен быть обернут в асинхронные функции (`async function`).
- Обязательное использование `try...catch` для обработки ошибок.
- Использовать `path.join(__dirname, ...)` для путей

Задание 1: Создание структуры. Напишите асинхронную функцию `setup()`, которая проверяет наличие папки `storage`. Если её нет — создает её. После создания запишите в файл `storage/status.txt` строку "Система готова".

```
1 async function setup() {
2   fs.mkdir(path.join('storage'), { recursive: true }, (err) => {
3     if (err) console.log(err);
4     return;
5   });
6   fs.writeFile(path.join('storage', 'status.txt'), 'Система готова', 'utf-8')
7 } setup();
```

Задание 2: Асинхронное чтение. Создайте файл `tasks.txt` вручную. Напишите функцию, которая асинхронно читает этот файл. Если файл прочитан успешно — выведите его содержимое в верхнем регистре. Если файла нет — выведите "Файл задач не найден", не роняя программу.

```
1 async function readUpper() {
2   try {
3     let content = await fs.readFile(
4       path.join(__dirname, 'tasks.txt'),
5       'utf-8'
6     );
7     console.log(content.toUpperCase());
8   } catch (err) { console.log(err) }
9 } readUpper();
```

Задание 3: Логирование с меткой времени. Реализуйте функцию `addLog(message)`, которая асинхронно дописывает (append) в файл `storage/activity.log` строку с текущей датой и переданным сообщением.



```

1  async function addLog(message) {
2      try {
3          await fs.writeFile(
4              path.join(__dirname, 'storage', 'activity.log'),
5              '\n' + message,
6              { flag: "a" },
7              'utf-8'
8          );
9      } catch (err) { console.log(err) }
10 } addLog("test");

```

Задание 4: Цепочка операций. Напишите функцию, которая:

- Читает файл source.txt.
- Создает копию этого файла с названием copy.txt.
- После успешного копирования удаляет оригинал (source.txt).



```

1  async function makeCopy() {
2      try {
3          let content = await fs.readFile(
4              path.join(__dirname, 'source.txt'),
5              'utf-8'
6          );
7          await fs.unlink(path.join(__dirname, 'source.txt'))
8          await fs.writeFile(
9              path.join(__dirname, 'copy.txt'),
10             content,
11             'utf-8'
12         );
13     } catch (err) { console.log(err) }
14 } makeCopy();

```

Задание 5: Параллельная проверка (Promise.all). Дополнительное задание: Создайте массив имен файлов. Используя Promise.all и метод fs.access (или fs.stat), проверьте существование всех этих файлов одновременно и выведите результат.

```
1  async function checkFiles() {
2    let fileList = [
3      'copy.txt',
4      'index.js',
5      'tasks.txt'
6    ];
7
8    let checked = fileList.map(async (file) => {
9      try {
10        fs.access(path.join(__dirname, file));
11
12        return {
13          file,
14          exists: true
15        };
16      } catch (err) {
17        return {
18          file,
19          exists: false
20        };
21      }
22    });
23
24    const result = await Promise.all(checked);
25    result.forEach(({file, exists}) => {
26      console.log(`${file}: ${exists ? "существует" : "не существует"}`);
27    });
28 } checkFiles();
```